Descripción del código y estrategias de paralelización

Cómputo Paralelo y en la Nube — Proyecto Apertura: DBSCAN paralelo para la detección de ruido con OpenMP. Autor: Jorge Daniel Lafarga Axotla. Código: `src/dbscan.cpp`.

1. Problema

Dado un conjunto de puntos en el plano, cada punto se clasifica con dos parámetros: ε (distancia) y `min_samples` (número mínimo de puntos a distancia ≤ ε para considerar densa una zona). Se usan los del notebook del curso: ε = 0.03 y `min_samples` = 10 (el propio punto cuenta, igual que en sklearn).

Etiquetas de salida:

- **Core (1.er orden), etiqueta 1:** tiene al menos `min_samples` puntos a distancia ≤ ε.
- **Core de 2.º orden, etiqueta 2:** no es core, pero tiene un core a distancia ≤ ε.
- **Ruido, etiqueta 0:** el resto.

Solo se detecta ruido: no se asignan identificadores de cluster, así que no hace falta expandir clusters ni unir componentes conexas.

La distancia euclidiana $d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$ se evalúa como $d^2 \le \varepsilon^2$, que es equivalente y evita la raíz.

2. Organización del código

Elemento	Función
`marcar_cores`	Fase 1: decide si cada punto propio de una región es core.
`etiquetar`	Fase 2: asigna core, core de 2.º orden o ruido.
`dbscan_serial`	Procesa todos los puntos en una sola región, en forma serial.
`dbscan_p1`	Procesa todos los puntos en una sola región, repartiendo los puntos entre hilos.
`dbscan_particionado`	Núcleo de P2: divide el plano en `gx × gy` regiones, las procesa en paralelo y mantiene la consistencia.
`dbscan_malla`	Configura la malla de `k` regiones usada por P2.
`construir_region`	Reúne los puntos de una celda y su halo.
`bench`	Experimento completo (sección 6).

Las tres versiones comparten los mismos kernels (`marcar_cores` y `etiquetar`). En P1, los bucles por punto de ambos kernels se ejecutan con OpenMP; en la serial y dentro de cada región de P2, esos bucles son seriales.

3. Versión serial

Construye una única región que contiene todos los puntos, ejecuta la fase 1 y luego la fase 2. En la fase 1 el conteo de vecinos de un punto se detiene al llegar a `min_samples` (un punto core no necesita más). En la fase 2 la búsqueda de un core vecino se detiene en cuanto aparece uno. Un punto de ruido sí recorre todos los puntos en ambas fases.

4. P1: versión paralela sin partición

P1 construye una única región con todos los puntos, como la versión serial. La fase 1 reparte los puntos entre hilos con `#pragma omp parallel for schedule(dynamic, 64)` para determinar cuáles son core. Al terminar ese bucle, todos los valores de `es_core` están disponibles. La fase 2 reparte de nuevo los puntos para asignar las etiquetas. Cada hilo escribe una posición distinta de los arreglos de resultados, de acuerdo con el índice del punto.

P1 compara cada punto con candidatos de toda la entrada; no crea celdas ni halos. Esto permite medir el beneficio de OpenMP sobre el mismo trabajo de búsqueda que realiza la serial.

5. P2: versión paralela con partición

5.1 Partición

Con los valores `k = {2, 4, 8, 16, 32, 64}` usados en el experimento, el plano (caja que contiene a los puntos) se divide en una malla de `gx × gy` celdas con `gx·gy = k`. Se elige `gx` como la menor potencia de 2 tal que `gx² ≥ k` y `gy = k/gx`. Las dimensiones resultantes son 2×1, 2×2, 4×2, 4×4, 8×4 y 8×8, respectivamente.

La implementación actual de la malla está diseñada para esos valores de `k`, que son potencias de 2.

Cada punto pertenece a una sola celda (su dueño), calculada en un `parallel for`.

5.2 ¿Cómo unir los resultados para mantener consistencia?

El problema de dividir la matriz es que un punto cerca de una frontera tiene vecinos en la región contigua. Se resuelve con dos mecanismos:

1. **Halo de ancho ε.** Cada región trabaja con una lista de candidatos formada por sus puntos propios y todos los puntos de otras celdas que estén a distancia ≤ ε de su rectángulo (`construir_region`). El margen usado es $\varepsilon + 10^{-9}$ para absorber redondeo.
2. **Dos fases separadas por una barrera.** En la fase 1 cada región calcula si sus puntos propios son core y lo escribe en un arreglo global `es_core`. Al terminar el `parallel for` hay una barrera implícita; después, en la fase 2, cada región consulta `es_core` (global) para decidir el re-etiquetado.

Por qué el resultado es idéntico al serial. Sea p un punto propio de una región y q cualquier punto con $|p - q| \le \varepsilon$. Como p está dentro del rectángulo de su región, la distancia de q a ese rectángulo es ≤ ε, así que q está en la lista de candidatos. Por lo tanto:

- el conteo de vecinos de p en la fase 1 es el mismo que en la serial, y `es_core[p]` también;
- en la fase 2, todos los vecinos de p están en la lista y su valor de `es_core` ya es definitivo gracias a la barrera.

No se necesita una etapa de "unión" posterior: cada punto se etiqueta una sola vez, por su dueño, con información completa.

5.3 Uso de OpenMP

- `#pragma omp parallel for` para calcular la celda de cada punto.
- `#pragma omp parallel for schedule(dynamic, 1)` sobre las regiones, en ambas fases. Las regiones tienen costos muy distintos (los clusters son densos y el resto casi está vacío), por lo que el reparto dinámico equilibra la carga mejor que uno estático.
- **Sin condiciones de carrera:** en la fase 1 cada elemento de `es_core` lo escribe un solo hilo (su dueño) y en la fase 2 se lee, junto con `etiqueta`, que también tiene un solo escritor por elemento. Se usan arreglos separados (`es_core` y `etiqueta`) para que en la fase 2 nadie lea una posición que otro hilo esté escribiendo.
- Cada región es un objeto independiente (`Region`) con copias contiguas de sus coordenadas, lo que mejora la localidad de caché.

5.4 Costos de la partición

Construir la región de una celda recorre los n puntos, así que construir todas cuesta O(k·n) (repartido entre los hilos). A cambio, cada región compara contra una lista de candidatos mucho más corta que n. Por eso aumentar `k` primero ayuda y después deja de ayudar (ver el documento de evaluación). Al crecer `k`, también aumenta el trabajo de construir regiones y copiar puntos del halo.

6. Entrada y salida

- **Entrada:** `.csv` con dos columnas `x,y` sin encabezado, generado con la libreta `DBSCAN_noise.ipynb`.
- **Salida:** `<N>_results.csv` con columnas `x,y,etiqueta`, en la misma carpeta que la entrada, que se visualiza con la misma libreta.

7. Verificación y experimento

- El modo `--bench` mide la serial, P1 con un solo valor `k = 1` y P2 con seis valores de `k`. Registra cada iteración en `results/experimentos.csv` y compara las etiquetas con las de la versión serial. Las 2,320 corridas del experimento actualizado terminaron sin diferencias.
- En una verificación anterior se comparó el conjunto de puntos de ruido contra `sklearn.cluster.DBSCAN` (ε = 0.03, `min_samples` = 10) con 4,000 y 20,000 puntos: no hubo diferencias.

8. Compilación y uso

```
make
build/dbscan data/4000_data.csv p1 1 8
build/dbscan data/4000_data.csv malla 4 8
build/dbscan --bench data results/experimentos.csv
```